

Sistema de Gestão de Locações de Imóveis

RCP Data Imob

Documentação Técnica Completa

Disciplina: Laboratório de Programação 3

Professor: Matheus Vanzan

Integrantes: Marcell Parra Araújo B. Silva, Rafael Vargas Carvalheira, Ruan Pablo Rodrigues

Data: Março de 2026

1. Apresentação do Projeto

O projeto Sistema de Gestão de Locações de Imóveis consiste em uma aplicação web desenvolvida para apoiar a administração de imóveis destinados à locação. A proposta central do sistema é reunir, em uma única plataforma, as operações de cadastro de imóveis e clientes, controle de reservas, verificação de disponibilidade e acompanhamento financeiro da operação.

A solução foi concebida em arquitetura cliente-servidor desacoplada, com backend em API REST e frontend em SPA (Single Page Application). Essa separação facilita a manutenção do sistema, melhora a organização do código e permite reutilização futura da API por aplicações mobile.

2. Objetivo do Sistema

O objetivo do sistema é oferecer uma solução completa para o gerenciamento de locações de imóveis, permitindo:

- Cadastrar e manter o portfólio de imóveis disponíveis
- Cadastrar clientes e seus dados de contato
- Criar, consultar, editar e cancelar locações
- Verificar disponibilidade de imóveis com base em período e restrições de data
- Categorizar imóveis por características relevantes
- Registrar receitas e despesas associadas à operação
- Acompanhar indicadores financeiros e operacionais

3. Escopo Funcional

3.1. Gestão de Imóveis com Categorias

- Cadastro completo com título, descrição, endereço, cidade, estado, CEP, quartos, banheiros, vagas de garagem, área, valor de aluguel e valor de venda
- Associação de múltiplas categorias por imóvel (Piscina, Ar-condicionado, Pet-friendly)
- Busca e filtros por título, cidade e status de disponibilidade
- Visualização em cards com informações resumidas
- Edição e exclusão com validações de integridade

3.2. Sistema de Locações com Controle de Status

- Listagem de locações com imóvel, cliente, datas, valor mensal e status (ativa/inativa)
- Formulário de nova locação com selects dinâmicos
- Encerramento de locações ativas via PATCH
- Indicação visual de locação ativa vs. inativa com badges semânticos

3.3. Controle Financeiro com Dashboard Analítico

- Registro de lançamentos financeiros com tipo (receita/despesa), valor, data de vencimento
- Filtros por status (pendente/pago/atrasado)
- Cards de resumo com total de receitas e despesas
- Dashboard com KPIs consolidados e gráfico de evolução financeira dos últimos 6 meses

4. Funcionalidade Selecionada para o Mobile

A funcionalidade escolhida para adaptação em ambiente mobile foi a **Busca de Disponibilidade e Criação de Reserva**.

Recursos previstos:

- Tela com campos de data de entrada e saída para busca rápida
- Listagem de imóveis disponíveis no período informado
- Exibição resumida com tipo, capacidade e valor estimado
- Tela de detalhes do imóvel com categorias
- Confirmação de reserva com seleção de cliente
- Feedback imediato em caso de sucesso ou conflito de datas

Justificativa: A consulta de disponibilidade e a confirmação de reservas são tarefas que exigem agilidade. Uma versão mobile desse fluxo reduz a dependência de computador e acelera o atendimento ao cliente.

5. Arquitetura da Solução

A aplicação adota uma arquitetura dividida em três camadas principais:

- **Banco de dados:** responsável pelo armazenamento persistente
- **Backend:** responsável pela lógica de negócio, exposição dos endpoints e comunicação com o banco
- **Frontend:** responsável pela interface visual e interação com o usuário

Os serviços são orquestrados com Docker Compose.

5.1. Serviços e Portas

Serviço	Imagem / Tecnologia	Porta	Descrição
db	postgres:13-alpine	5432	Banco de dados PostgreSQL
backend	node:18-alpine (build)	3000	API REST Node.js / Express
frontend	node:20-alpine (build)	5173	Interface React / Vite

6. Stack Tecnológica

6.1. Backend

Tecnologia	Versão	Função no projeto
Node.js	v18+	Runtime JavaScript no servidor
Express.js	4.18.2	Framework para API REST
PostgreSQL	v13+	Banco de dados relacional principal
pg	8.11.3	Driver PostgreSQL para Node.js
cors	2.8.5	Middleware para Cross-Origin Requests

6.2. Frontend

Tecnologia	Versão	Função no projeto
React	v18.2	Construção da interface SPA
Vite	v6.0	Build tool e servidor de desenvolvimento
React Router DOM	v7.0	Roteamento client-side
Tailwind CSS	v3.4	Estilização responsiva com utility-first
Recharts	v2.12	Gráficos do dashboard financeiro
Lucide React	v0.400	Biblioteca de ícones

Tecnologia	Versão	Função no projeto
Axios	v1.7	Comunicação HTTP com a API

6.3. Infraestrutura

Tecnologia	Função
Docker	Containerização dos serviços
Docker Compose	Orquestração multi-container
Google Fonts	Tipografia (Plus Jakarta Sans + DM Sans)

7. Modelo de Dados

Tabelas principais:

Tabela	Conteúdo
usuarios	Dados de acesso e perfil dos administradores
imoveis	Cadastro dos imóveis com atributos, valores e status
categorias_imoveis	Categorias personalizadas para classificação
imovel_categorias	Relacionamento N:N entre imóveis e categorias
clientes	Cadastro de locatários
locacoes	Contratos de locação, datas, valores e status
financeiro	Lançamentos de receitas e despesas

7.1. Detalhamento — Tabela imoveis

Coluna	Tipo	Restrições	Descrição
id	SERIAL	PRIMARY KEY	Identificador único
titulo	VARCHAR(255)	NOT NULL	Título do imóvel
descricao	TEXT	—	Descrição detalhada
endereco	VARCHAR(500)	NOT NULL	Endereço completo
cidade	VARCHAR(100)	—	Cidade
estado	VARCHAR(2)	—	UF
cep	VARCHAR(10)	—	CEP
valor_aluguel	NUMERIC(12,2)	—	Valor mensal de aluguel
valor_venda	NUMERIC(12,2)	—	Valor de venda
area	NUMERIC(10,2)	—	Área em m²
quartos	INTEGER	DEFAULT 0	Número de quartos
banheiros	INTEGER	DEFAULT 0	Número de banheiros
vagas_garagem	INTEGER	DEFAULT 0	Vagas de garagem
disponivel	BOOLEAN	DEFAULT TRUE	Disponível para locação
criado_em	TIMESTAMP	DEFAULT NOW()	Data de cadastro

7.2. Detalhamento — Tabela locacoes

Coluna	Tipo	Restrições	Descrição
id	SERIAL	PRIMARY KEY	Identificador único
imovel_id	INTEGER	FK → imoveis(id)	Imóvel locado
cliente_id	INTEGER	FK → clientes(id)	Locatário
data_inicio	DATE	NOT NULL	Início do contrato
data_fim	DATE	—	Fim do contrato
valor_mensal	NUMERIC(12,2)	NOT NULL	Valor mensal
ativa	BOOLEAN	DEFAULT TRUE	Locação em vigor
criado_em	TIMESTAMP	DEFAULT NOW()	Data de registro

8. Estrutura do Backend

8.1. Organização do Projeto

Caminho	Descrição
backend/src/index.js	Ponto de entrada; inicializa o Express e registra as rotas
backend/src/db/pool.js	Pool de conexão com PostgreSQL
backend/src/routes/imoveis.js	Rotas CRUD de imóveis + disponibilidade (F1)
backend/src/routes/clientes.js	Rotas CRUD de clientes
backend/src/routes/locacoes.js	Rotas CRUD de locações
backend/src/routes/categorias.js	Rotas CRUD de categorias
backend/src/routes/financeiro.js	Rotas CRUD de lançamentos financeiros
backend/src/routes/status.js	Endpoint de healthcheck

9. API REST

A API foi construída com Node.js e Express, seguindo princípios REST. O servidor escuta na porta 3000.

9.1. Endpoint de Healthcheck

Método	Rota	Descrição
GET	/status	Verifica status da aplicação e do banco

9.2. Endpoints de Imóveis

Método	Rota	Descrição
GET	/imoveis	Lista todos os imóveis cadastrados
GET	/imoveis/disponibilidade	Lista imóveis livres em período (F1) com categorias
GET	/imoveis/:id	Retorna um imóvel específico pelo ID
POST	/imoveis	Cria um novo imóvel
PUT	/imoveis/:id	Atualiza os dados de um imóvel
DELETE	/imoveis/:id	Remove um imóvel do sistema

9.3. Endpoints de Clientes

Método	Rota	Descrição
GET	/clientes	Lista todos os clientes cadastrados
GET	/clientes/:id	Retorna um cliente específico pelo ID
POST	/clientes	Cadastra um novo cliente
PUT	/clientes/:id	Atualiza os dados de um cliente
DELETE	/clientes/:id	Remove um cliente do sistema

9.4. Endpoints de Locações

Método	Rota	Descrição
GET	/locacoes	Lista todas as locações registradas
GET	/locacoes/:id	Retorna uma locação específica
POST	/locacoes	Cria um novo contrato de locação
PATCH	/locacoes/:id/encerrar	Encerra uma locação ativa

9.5. Endpoints de Categorias

Método	Rota	Descrição
GET	/categorias	Lista todas as categorias disponíveis
POST	/categorias	Cria uma nova categoria
DELETE	/categorias/:id	Remove uma categoria do sistema

9.6. Endpoints Financeiros

Método	Rota	Descrição
GET	/financeiro	Lista todos os lançamentos financeiros
POST	/financeiro	Registra um novo lançamento
PATCH	/financeiro/:id/pagar	Registra pagamento de um lançamento

9.7. Resumo Geral de Endpoints

Entidade	GET	POST	PUT	PATCH	DELETE	Total
/status	1	—	—	—	—	1
/imoveis	3	1	1	—	1	6
/clientes	2	1	1	—	1	5

Entidade	GET	POST	PUT	PATCH	DELETE	Total
/locacoes	2	1	—	1	—	4
/categorias	1	1	—	—	1	3
/financeiro	1	1	—	1	—	3
TOTAL	10	5	2	2	3	22

Nota: O terceiro GET de /imoveis é o endpoint /imoveis/disponibilidade implementado na Funcionalidade F1.

10. Banco de Dados e Docker — Entregável 3

O Entregável 3 consolidou a configuração do banco de dados PostgreSQL via Docker, modelagem das entidades e integração completa entre o backend e o banco.

10.1. Status dos Requisitos

Requisito	Status
Banco de dados configurado no Docker (PostgreSQL 13)	Concluído
Modelagem das entidades e relacionamentos	Concluído
Backend integrado ao banco com persistência via API	Concluído
CRUD completo testado via Postman	Concluído
Documentação e capturas de tela	Concluído

10.2. Testes CRUD via Postman

Operação	Método	Endpoint	Status Retornado
Create	POST	/clientes	201 Created
Read (listar)	GET	/clientes	200 OK
Update	PUT	/clientes/:id	200 OK
Delete	DELETE	/clientes/:id	200 OK

O endpoint **GET /status** executa um **SELECT NOW()** contra o banco, confirmando que a conexão está ativa. Retorna HTTP 200 com banco conectado ou HTTP 503 em caso de falha.

11. Frontend e Docker — Entregável 4

O Entregável 4 implementou o frontend completo do sistema com React, configurado e executável via Docker.

11.1. Requisitos Atendidos

Requisito	Status
Frontend configurado e rodando no Docker	Concluído
Tela inicial do sistema criada (Dashboard)	Concluído
Navegação mínima entre telas (menu/rotas)	Concluído
Frontend consumindo dados da API (múltiplas chamadas)	Concluído

Requisito	Status
Exibição de dados vindos da API (listas, KPIs, gráficos)	Concluído
Documentação correspondente e capturas de tela	Concluído

11.2. Telas Implementadas

Rota	Tela	Funcionalidades
/	Dashboard	4 KPIs, gráfico de barras (Recharts) com 6 meses, últimas locações
/imoveis	Imóveis	Cards com filtros (cidade/status/busca), CRUD completo via modal
/clientes	Cientes	Tabela com busca por nome, CRUD completo via modal
/locacoes	Locações	Tabela com selects de imóvel/cliente, encerramento de locação
/financeiro	Financeiro	Cards receitas/despesas, filtro por status, ação de pagamento

11.3. Consumo da API — Entregável 4

Tela	Endpoints consumidos
Dashboard	GET /imoveis, GET /clientes, GET /locacoes, GET /financeiro
Imóveis	GET /imoveis, POST /imoveis, PUT /imoveis/:id, DELETE /imoveis/:id
Cientes	GET /clientes, POST /clientes, PUT /clientes/:id, DELETE /clientes/:id
Locações	GET /locacoes, GET /imoveis, GET /clientes, POST /locacoes, PATCH /locacoes/:id/encerrar
Financeiro	GET /financeiro, GET /locacoes, POST /financeiro, PATCH /financeiro/:id/pagar

12. Funcionalidade F1 — Busca de Disponibilidade e Criação de Reserva (Entregável 5)

12.1. Descrição

O Entregável 5 implementou a Funcionalidade F1 completa: **Busca de Disponibilidade e Criação de Reserva**. Esta funcionalidade permite ao usuário buscar imóveis disponíveis em um período específico e criar reservas diretamente pela interface web.

12.2. Requisitos Atendidos

Requisito	Status
Endpoint da API para busca de disponibilidade implementado	Concluído
Lógica de conflito de datas no banco	Concluído
Retorno de categorias do imóvel no endpoint	Concluído
Tela de busca com campos de período	Concluído
Listagem de imóveis disponíveis em cards	Concluído
Modal de confirmação de reserva com seleção de cliente	Concluído
Feedback de sucesso e atualização da listagem	Concluído
Fluxo completo funcionando no Docker	Concluído

12.3. Endpoint Implementado

Método	Rota	Parâmetros	Descrição
GET	/imoveis/disponibilidade	data_inicio, data_fim (query)	Lista imóveis disponíveis no período com categorias

Lógica de conflito de datas:

Um imóvel é considerado **INDISPONÍVEL** se existir uma locação ativa (ativa = true) cujo período se sobrepõe ao período solicitado. A condição de sobreposição é verificada por:

```
NOT EXISTS (  
  SELECT 1 FROM locacoes l  
  WHERE l.imovel_id = i.id  
  AND l.ativa = true  
  AND l.data_inicio <= data_fim  
  AND (l.data_fim IS NULL OR l.data_fim >= data_inicio)  
)
```

Isso garante que:

- Locações sem data_fim (abertas) bloqueiam o imóvel indefinidamente

- Qualquer sobreposição parcial de período também bloqueia
- Apenas locações ativas (ativa = true) são consideradas no bloqueio

12.4. Nova Tela Implementada

Rota	Tela	Funcionalidades
/disponibilidade	Disponibilidade	Form de busca por período, cards de resultados com categorias, modal de reserva

12.5. Fluxo Completo da F1

1. O usuário acessa a tela "Disponibilidade" via sidebar (ícone CalendarSearch)
2. Preenche Data de Entrada e Data de Saída e clica em "Buscar Disponíveis"
3. O frontend chama GET /imoveis/disponibilidade?data_inicio=&data_fim=
4. Imóveis disponíveis são exibidos em cards contendo: título, cidade/estado, quartos, banheiros, vagas de garagem, área, categorias como badges coloridos e valor mensal
5. O usuário clica em "Reservar" em um imóvel
6. Modal de confirmação é exibido com: resumo do imóvel, período pré-preenchido (readonly), select de cliente, campo de valor mensal editável
7. Ao confirmar, o frontend chama POST /locacoes criando a locação no banco
8. Banner de sucesso é exibido e o imóvel reservado é removido da listagem
9. Em caso de erro (ex: conflito), mensagem de erro aparece no modal

12.6. Arquivos Criados/Modificados

Arquivo	Operação	Descrição
backend/src/routes/imoveis.js	Modificado	Adicionado endpoint GET /disponibilidade com lógica SQL de conflito
frontend/src/pages/Disponibilidade.jsx	Criado	Nova página com form de busca, cards e modal de reserva
frontend/src/App.jsx	Modificado	Rota /disponibilidade adicionada
frontend/src/components/Layout.jsx	Modificado	Item "Disponibilidade" com ícone CalendarSearch na sidebar

12.7. Testes Realizados

Teste	Esperado	Resultado
GET /disponibilidade sem params	HTTP 400	Passou
Busca em período livre (2 imóveis disponíveis)	2 imóveis	Passou
Busca em período com imóvel ocupado	1 imóvel (conflito detectado)	Passou
Categorias retornadas no JSON	Array categorias	Passou

Teste	Esperado	Resultado
Criar reserva via POST /locacoes	Locação criada ativa=true	Passou
Busca após reserva (imóvel reservado some)	1 imóvel removido	Passou
Endpoint via proxy Vite /api/...	JSON com categorias	Passou
Frontend acessível em localhost:5173	HTTP 200	Passou

13. Execução do Ambiente

Para executar o projeto completo:

Subir todos os serviços:

```
docker compose up --build -d
```

Verificar status dos containers:

```
docker compose ps
```

Ver logs de um serviço:

```
docker compose logs backend
```

```
docker compose logs frontend
```

Parar os serviços:

```
docker compose down
```

13.1. Acessos

Serviço	URL
Frontend	http://localhost:5173
API REST	http://localhost:3000
PostgreSQL	localhost:5432

13.2. Telas Disponíveis

Rota	Tela
/	Dashboard — KPIs e gráfico financeiro
/imoveis	Gestão de Imóveis
/clientes	Gestão de Clientes
/locacoes	Gestão de Locações
/financeiro	Controle Financeiro
/disponibilidade	Busca de Disponibilidade e Reserva (F1)

14. Considerações sobre a Implementação

A proposta do sistema demonstra uma evolução consistente ao longo dos entregáveis:

- **No Entregável 0**, foi definida a proposta do sistema, seu escopo funcional, a stack e a visão geral das telas
- **No Entregável 2**, foi consolidada a implementação da API REST, com organização modular, integração com PostgreSQL e endpoints CRUD por entidade
- **No Entregável 3**, foi configurado o banco de dados PostgreSQL via Docker, com modelagem completa das entidades, inicialização automática do schema e validação de CRUD via Postman
- **No Entregável 4**, foi implementado o frontend completo com React, Vite e Tailwind CSS, configurado no Docker com proxy para o backend, consumindo dados reais da API em todas as 5 telas do sistema
- **No Entregável 5**, foi implementada a Funcionalidade F1 completa (Busca de Disponibilidade e Criação de Reserva), com novo endpoint de API com lógica de conflito de datas, nova tela no frontend com busca por período e fluxo de reserva end-to-end funcionando no Docker

15. Conclusão

O Sistema de Gestão de Locações de Imóveis (RCP Data Imob) foi projetado e implementado como uma solução web completa para atender às necessidades de uma administradora de imóveis. O sistema integra cadastro, locações, categorização e controle financeiro em uma única aplicação com interface visual profissional.

Do ponto de vista técnico, a solução utiliza tecnologias atuais e amplamente adotadas no desenvolvimento web, com separação clara entre frontend, backend e banco de dados. A utilização de Docker Compose contribui para a reprodutibilidade do ambiente.

O Entregável 5 adicionou ao sistema a funcionalidade mais estratégica para uso mobile — a Busca de Disponibilidade e Criação de Reserva —, demonstrando capacidade de implementar fluxos completos end-to-end com lógica de negócio no banco de dados (verificação de conflito de datas), endpoint dedicado na API e interface web completa com feedback ao usuário.

16. Referências Internas do Projeto

- Entregável 0 — Definição do tema, escopo, stack e telas
- Entregável 2 — Implementação da API REST e organização do backend
- Entregável 3 — Banco de dados PostgreSQL + Docker + testes CRUD via Postman
- Entregável 4 — Frontend React + Vite + Tailwind CSS no Docker, consumindo API
- Entregável 5 — Funcionalidade F1: Busca de Disponibilidade e Criação de Reserva
- Coleção Postman — `postman/RCP_Data_Imob.postman_collection.json`